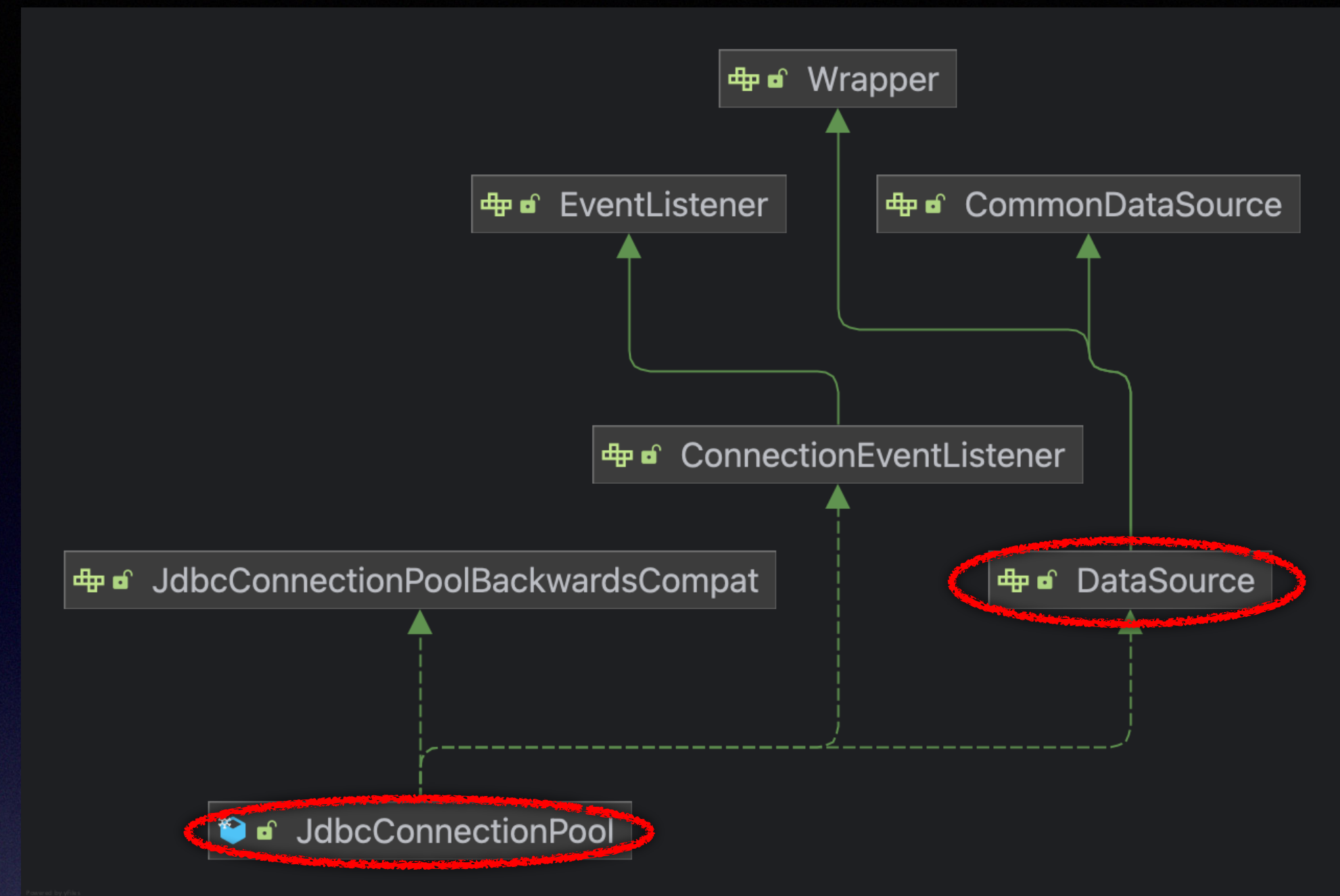


HikariCP 내부 코드 파헤치기

스타크

H2에서의 Connection Pool



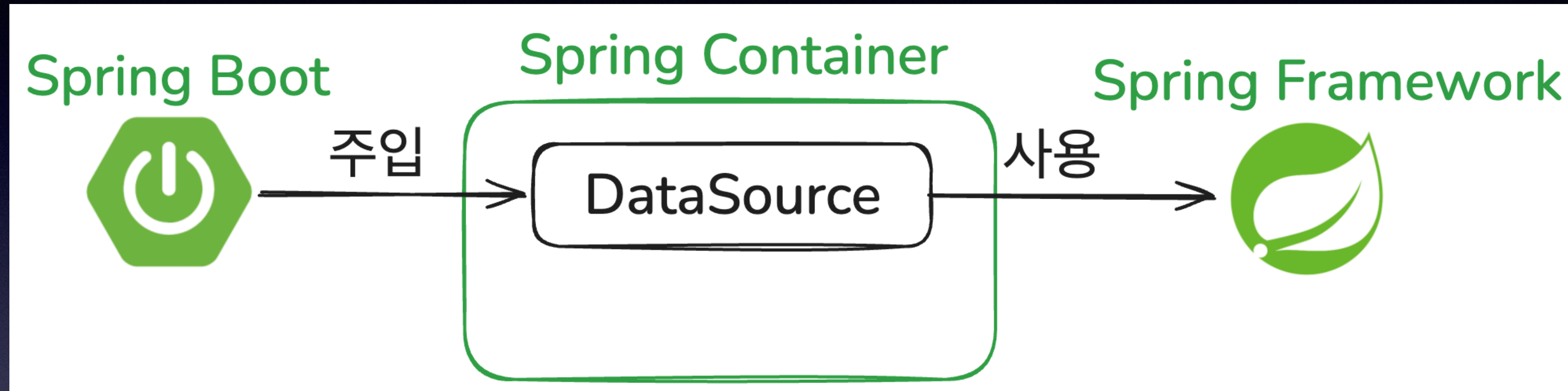
```
public final class JdbcConnectionPool
    implements DataSource, ConnectionEventListener, JdbcConnectionPoolBackwardsCompat {

    private static final int DEFAULT_TIMEOUT = 30;
    private static final int DEFAULT_MAX_CONNECTIONS = 10;

    private final ConnectionPoolDataSource dataSource;
    private final Queue<PooledConnection> recycledConnections = new ConcurrentLinkedQueue<>();
    private PrintWriter logWriter;
    private volatile int maxConnections = DEFAULT_MAX_CONNECTIONS;
    private volatile int timeout = DEFAULT_TIMEOUT;
    private AtomicInteger activeConnections = new AtomicInteger();
    private AtomicBoolean isDisposed = new AtomicBoolean();
```


레벨2 에서부터 배울 Spring은
어떻게 Connection Pool을 사용할까??

Spring Framework와 Spring Boot 역할 구분하기

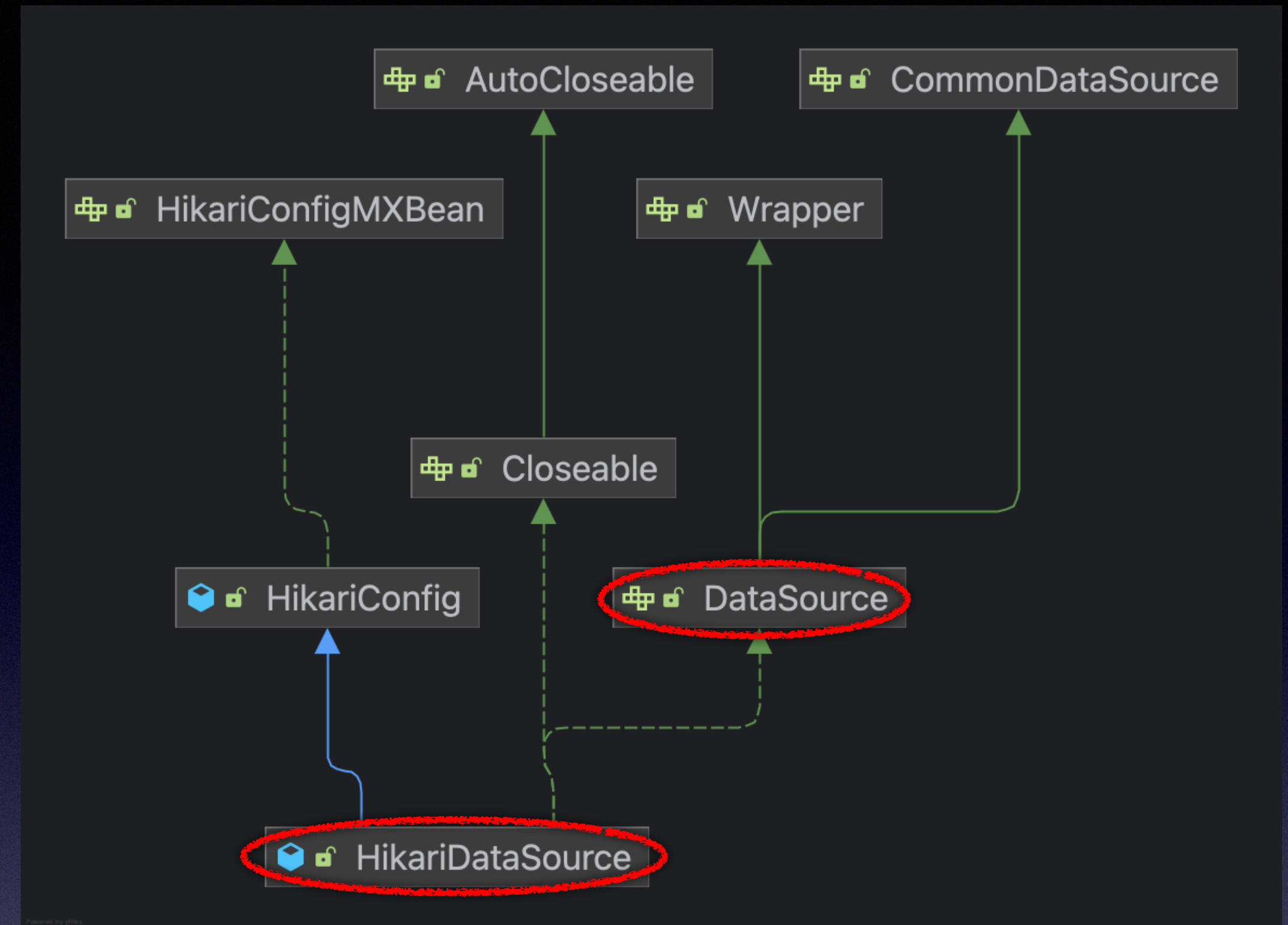


- Spring Framework는 Spring Container에 등록되어있는 DataSource Bean을 주입받아 사용하는 역할
- Spring Boot는 Spring Framework가 사용할 수 있게 DataSource Bean을 자동으로 생성해서 등록해주는 역할

Spring Boot가 선택한 DataSource

- Spring Boot는 버전 2.0부터 기본값으로 HikariCP 라이브러리를 사용하기 시작
 - 기본값일 뿐 얼마든지 변경도 가능
- Connection Pool의 실제 동작은 Spring Framework, Spring Boot가 아닌!
HikariCP에서 수행

HikariCP 구조



```
public class HikariDataSource extends HikariConfig implements DataSource, Closeable
{
    private static final Logger LOGGER = LoggerFactory.getLogger(HikariDataSource.class);

    private final AtomicBoolean isShutdown = new AtomicBoolean();

    private final HikariPool fastPathPool;
    private volatile HikariPool pool;

    // ... 메서드 생략
}
```


HikariPool 살펴보기



```
public final class HikariPool extends PoolBase implements HikariPoolMXBean, IBagStateListener
{
    private final PoolEntryCreator poolEntryCreator = new PoolEntryCreator();
    private final PoolEntryCreator postFillPoolEntryCreator = new PoolEntryCreator("After adding ");
    private final ThreadPoolExecutor addConnectionExecutor;
    private final ThreadPoolExecutor closeConnectionExecutor;

    private final ConcurrentBag<PoolEntry> connectionBag;

    private final ScheduledExecutorService houseKeepingExecutorService;

    // 설명을 위한 필드만 남겨두고 나머지 필드, 메서드는 제거
}
```

커넥션이 담겨 있는 인스턴스 변수

ConcurrentBag 살펴보기

```
public class ConcurrentBag<T extends IConcurrentBagEntry> implements AutoCloseable
{
    private static final Logger LOGGER = LoggerFactory.getLogger(ConcurrentBag.class);

    private final CopyOnWriteArrayList<T> sharedList;
    private final boolean useWeakThreadLocals;

    private final ThreadLocal<List<Object>> threadLocalList;
    private final IBagStateListener listener;
    private final AtomicInteger waiters;
    private volatile boolean closed;

    private final SynchronousQueue<T> handoffQueue;
}
```

모든 스레드에서 공유되는 커넥션 저장소

각 스레드 별로 유지되는 커넥션 캐시

대기 중인 다른 스레드에게 커넥션을
즉시 직접 전달하기 위한(handoff) 공용 통로

PoolEntry 살펴보기

47가지의 상태값

```
public class ConcurrentBag<T extends IConcurrentBagEntry> implements AutoCloseable
{
    public interface IConcurrentBagEntry
    {
        int STATE_NOT_IN_USE = 0;
        int STATE_IN_USE = 1;
        int STATE_REMOVED = -1;
        int STATE_RESERVED = -2;

        boolean compareAndSet(int expectState, int newState);
        void setState(int newState);
        int getState();
    }

    // 그 외 생략
}
```

```
final class PoolEntry implements IConcurrentBagEntry
{
    private static final AtomicIntegerFieldUpdater<PoolEntry> stateUpdater;

    Connection connection;
    long lastAccessed;
    long lastBorrowed;

    @SuppressWarnings("FieldCanBeLocal")
    private volatile int state = 0;
    private volatile boolean evict;

    private final HikariPool hikariPool;

    private final boolean isReadOnly;
    private final boolean isAutoCommit;

    // 그 외 생략
}
```

자신을 관리하는 HikariPool의 참조

Connection의 요청과 반납 과정 살펴보기

- HikariDataSource의 초기화 직후에는 sharedList에만 커넥션이 담겨 있는 상태
- 이후 부터 Connection = PoolEntry 편의상 동일하게 부르겠습니다

1. Connection 요청 시

2. Connection 반납 시

Connection의 요청 시

- Connection을 요청하면 다음의 흐름으로 커넥션을 가져옵니다.

1. `HikariDataSource.getConnection()`
2. `HikariPool.getConnection()`
3. `ConcurrentBag.borrow()`

Connection의 요청 시 - HikariDataSource

```
public class HikariDataSource extends HikariConfig implements DataSource, Closeable
{
    private final HikariPool fastPathPool;
    private volatile HikariPool pool;

    @Override
    public Connection getConnection() throws SQLException
    {
        if (isClosed()) {
            throw new SQLException("HikariDataSource " + this + " has been closed.");
        }

        if (fastPathPool != null) {
            return fastPathPool.getConnection();
        }

        // See http://en.wikipedia.org/wiki/Double-checked\_locking#Usage\_in\_Java
        HikariPool result = pool;
        // HikariPool pool 변수 초기화 로직 생략

        return result.getConnection();
    }
}
```


Connection의 요청 시 - HikariPool



```
public Connection getConnection() throws SQLException
{
    return getConnection(connectionTimeout);
}
```



```
public Connection getConnection(final long hardTimeout) throws SQLException
{
    suspendResumeLock.acquire();
    final var startTime = currentTime();

    try {
        var timeout = hardTimeout;
        do {
            var poolEntry = connectionBag.borrow(timeout, MILLISECONDS);
            if (poolEntry == null) {
                break; // We timed out... break and throw exception
            }

            final var now = currentTime();
            if (poolEntry.isMarkedEvicted() || (elapsedMillis(poolEntry.lastAccessed, now) > aliveBypassWindowMs &&
isConnectionDead(poolEntry.connection))) {
                closeConnection(poolEntry, poolEntry.isMarkedEvicted() ? EVICTED_CONNECTION_MESSAGE : DEAD_CONNECTION_MESSAGE);
                timeout = hardTimeout - elapsedMillis(startTime);
            }
            else {
                metricsTracker.recordBorrowStats(poolEntry, startTime);
                if (isRequestBoundariesEnabled) {
                    try {
                        poolEntry.connection.beginRequest();
                    } catch (SQLException e) {
                        logger.warn("beginRequest Failed for: {}, ({})", poolEntry.connection, e.getMessage());
                    }
                }
            }
            return poolEntry.createProxyConnection(leakTaskFactory.schedule(poolEntry));
        } while (timeout > 0L);

        metricsTracker.recordBorrowTimeoutStats(startTime);
        throw createTimeoutException(startTime);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new SQLException(poolName + " - Interrupted during connection acquisition", e)
    } finally {
        suspendResumeLock.release();
    }
}
```


Connection의 요청 시 - ConcurrentBag

```
public T borrow(long timeout, final TimeUnit timeUnit) throws InterruptedException
{
    // Try the thread-local list first
    final var list = threadLocalList.get();
    for (var i = list.size() - 1; i >= 0; i--) {
        final var entry = list.remove(i);
        @SuppressWarnings("unchecked")
        final T bagEntry = useWeakThreadLocals ? ((WeakReference<T>) entry).get() : (T) entry;
        if (bagEntry != null && bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
            return bagEntry;
        }
    }

    // Otherwise, scan the shared list ... then poll the handoff queue
    final var waiting = waiters.incrementAndGet();
    try {
        for (T bagEntry : sharedList) {
            if (bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
                // If we may have stolen another waiter's connection, request another bag add.
                if (waiting > 1) {
                    listener.addBagItem(waiting - 1);
                }
                return bagEntry;
            }
        }
    }

    listener.addBagItem(waiting);

    timeout = timeUnit.toNanos(timeout);
    do {
        final var start = currentTime();
        final T bagEntry = handoffQueue.poll(timeout, NANOSECONDS);
        if (bagEntry == null || bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
            return bagEntry;
        }

        timeout -= elapsedNanos(start);
    } while (timeout > 10_000);

    return null;
}
finally {
    waiters.decrementAndGet();
}
}
```

굉장히 긴 코드...

⇒ return bagEntry 기준으로 코드 나누기

1. threadLocalList에서 커넥션 획득 시도

2. sharedList에서 커넥션 획득 시도

3. handoffQueue에서 커넥션 획득 시도

Connection의 요청 시 - ConcurrentBag

1. threadLocalList



```
// Try the thread-local list first
final var list = threadLocalList.get();
for (var i = list.size() - 1; i >= 0; i--) {
    final var entry = list.remove(i);
    @SuppressWarnings("unchecked")
    final T bagEntry = useWeakThreadLocals ? ((WeakReference<T>) entry).get() : (T) entry;
    if (bagEntry != null && bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
        return bagEntry;
    }
}
```

1. 현재 스레드에 캐싱된 커넥션 리스트 가져오기

2. 확인하는 커넥션들은
캐싱 목록에서 제거하며 가져옴

3. 가져온 PoolEntry가 null이 아니고??

PoolEntry의 상태를 STATE_NOT_IN_USE → STATE_IN_USE 로 변환에
성공 하면??

해당 PoolEntry 반환

Connection의 요청 시 - ConcurrentBag

1. threadLocalList

1. Connection 습득 실패 시 제거한 PoolEntry를 다시 캐싱하지 않는 이유???

- 원본 커넥션은 항상 sharedList에서 들고 있기 때문

2. PoolEntry의
compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE) 메서드 (이하 CAS 메서드)

- AtomicIntegerFieldUpdater를 사용한 원자적 연산
- 다른 스레드가 그 새 먼저 점유하게 되면 점유에 실패하게 됨.

Connection의 요청 시 - ConcurrentBag

2. sharedList

AtomicInteger 변수

```
final var waiting = waiters.incrementAndGet();
try {
    for (T bagEntry : sharedList) {
        if (bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
            // If we may have stolen another waiter's connection, request another bag add.
            if (waiting > 1) {
                listener.addBagItem(waiting - 1);
            }
            return bagEntry;
        }
    }
}
```

1. 커넥션 사용 대기자 수를 담은 변수값 증가 후
해당 값 가져오기

2. sharedList의 PoolEntry를 확인하며
CAS를 통해 상태 점유

3. 만약 커넥션 획득 후에 커넥션 대기자가 자신외에
또 존재한다면 listener에게 커넥션 추가 생성 요청

4. 점유에 성공한 PoolEntry 반환

Connection의 요청 시 - ConcurrentBag

2. sharedList

1. Listener에게 커넥션 추가를 요청할 때 이미 커넥션 풀에 커넥션이 최대 개수만큼 꽉 차 있으면??

```
private final class PoolEntryCreator implements Callable<Boolean>
{
    private synchronized boolean shouldContinueCreating() {
        return poolState == POOL_NORMAL && !Thread.interrupted() && getTotalConnections() < config.getMaximumPoolSize() &&
            (getIdleConnections() < config.getMinimumIdle() || connectionBag.getWaitingThreadCount() > getIdleConnections());
    }
}
```

AtomicInteger 타입 waiting 변수

1. 현재 전체 커넥션의 개수 < 최대 커넥션 풀 크기

2. 놀고 있는(idle) 커넥션 개수 < 설정값의 최소 idle 커넥션 개수

OR

idle 커넥션 개수 < 커넥션 요청 대기 스레드 개수

Connection의 요청 시 - ConcurrentBag

3. handoffQueue



```
final var waiting = waiters.incrementAndGet();  
try {  
    // sharedList에서 Connection 가져오는 코드...(생략)
```

1. 커넥션 보관함에서 커넥션 얻기 실패 상태
⇒ listener에게 커넥션 추가 생성 요청

```
listener.addBagItem(waiting);
```

2. 다른 스레드의 커넥션 반납까지 대기(timeout)
반납 시 해당 커넥션 습득

```
timeout = TimeUnit.NANOSECONDS.toNanos(timeout);
```

```
do {
```

```
    final var start = currentTime();
```

```
    final T bagEntry = handoffQueue.poll(timeout, NANOSECONDS);
```

```
    if (bagEntry == null || bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
```

```
        return bagEntry;
```

```
    }
```

3. 가져온 PoolEntry가 비어 있지 않고, CAS에 성공 시
해당 PoolEntry 반환

```
    timeout -= elapsedNanos(start);
```

```
    } while (timeout > 10_000);
```

```
    return null;
```

```
}
```

4. 정말 다 실패하면 null 반환

Connection의 요청 시 - ConcurrentBag

3. handoffQueue

1. handoffQueue 커넥션 습득 로직에서는 왜 ms가 아닌 ns??

```
final var waiting = waiters.incrementAndGet();
try {
    // sharedList에서 Connection 가져오는 코드...(생략)

    listener.addBagItem(waiting);

    timeout = timeUnit.toNanos(timeout);
    do {
        final var start = currentTime();
        final T bagEntry = handoffQueue.poll(timeout, NANOSECONDS);
        if (bagEntry == null || bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
            return bagEntry;
        }

        timeout -= elapsedNanos(start);
    } while (timeout > 10_000);

    return null;
}
```

남은 시간 기반 재시도 로직.
타이트하게 시간을 관리해서 극한의 효율을 추구하려고

Connection의 요청 시 - Back to ConcurrentBag

```
public T borrow(long timeout, final TimeUnit timeUnit) throws InterruptedException
{
    // Try the thread-local list first
    final var list = threadLocalList.get();
    for (var i = list.size() - 1; i >= 0; i--) {
        final var entry = list.remove(i);
        @SuppressWarnings("unchecked")
        final T bagEntry = useWeakThreadLocals ? ((WeakReference<T>) entry).get() : (T) entry;
        if (bagEntry != null && bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
            return bagEntry;
        }
    }

    // Otherwise, scan the shared list ... then poll the handoff queue
    final var waiting = waiters.incrementAndGet();
    try {
        for (T bagEntry : sharedList) {
            if (bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
                // If we may have stolen another waiter's connection, request another bag add.
                if (waiting > 1) {
                    listener.addBagItem(waiting - 1);
                }
                return bagEntry;
            }
        }
        listener.addBagItem(waiting);

        timeout = timeUnit.toNanos(timeout);
        do {
            final var start = currentTime();
            final T bagEntry = handoffQueue.poll(timeout, NANOSSECONDS);
            if (bagEntry == null || bagEntry.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
                return bagEntry;
            }

            timeout -= elapsedNanos(start);
        } while (timeout > 10_000);

        return null;
    } finally {
        waiters.decrementAndGet();
    }
}
```

캐시에서 습득하지 않은 경우! 커넥션 처리 후
마지막에는 waiters 변수의 값 감소

해당 메서드에서 시작한
커넥션 요청이 종료되었기 때문!!
(실제 PoolEntry이던, null 이던 일단 끝남)

Connection의 요청 시 - Back to HikariPool

```
public Connection getConnection(final long hardTimeout) throws SQLException
{
    suspendResumeLock.acquire();
    final var startTime = currentTime();

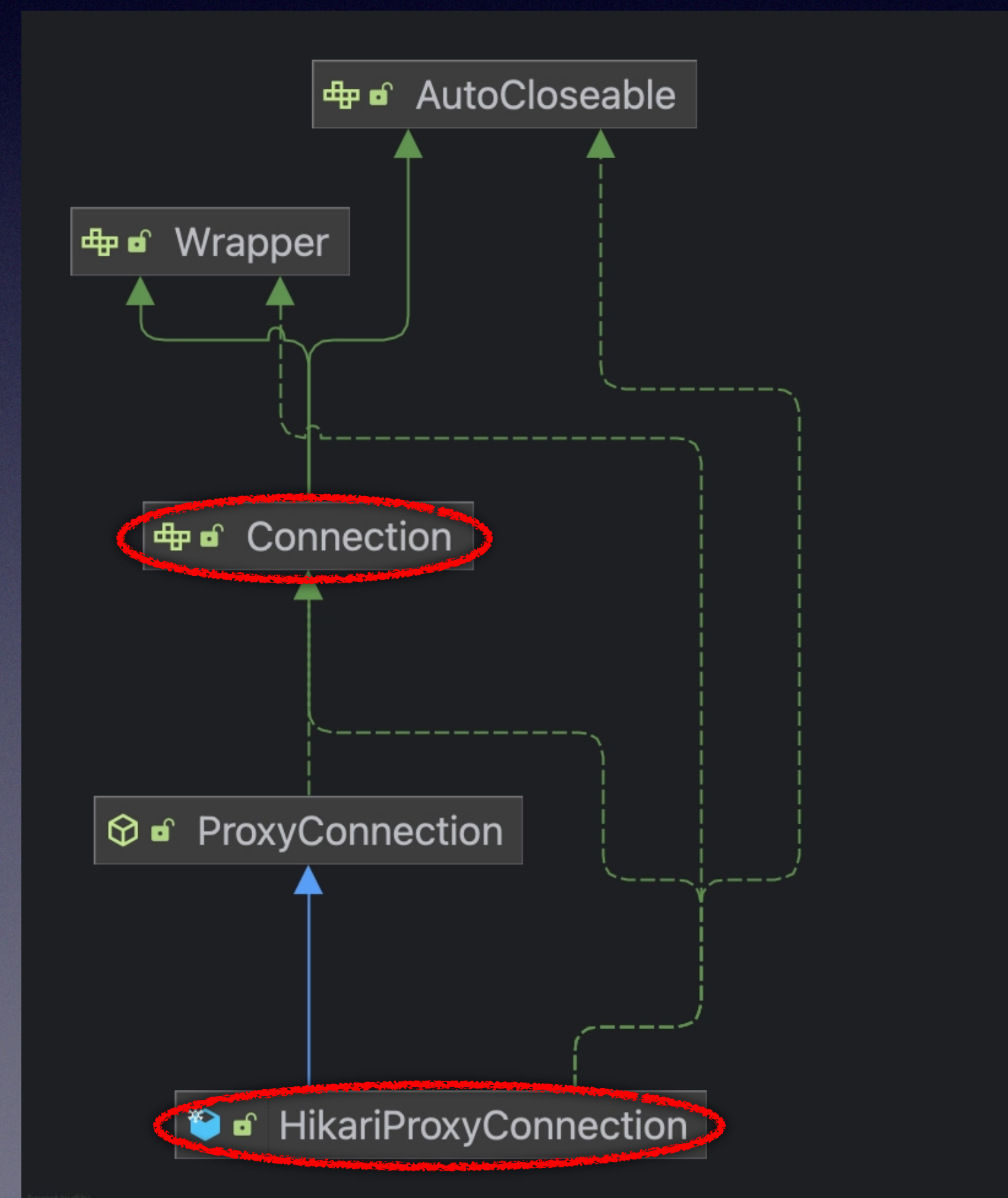
    try {
        var timeout = hardTimeout;
        do {
            var poolEntry = connectionBag.borrow(timeout, MILLISECONDS);
            if (poolEntry == null) {
                break; // We timed out... break and throw exception
            }

            final var now = currentTime();
            if (poolEntry.isMarkedEvicted() || (elapsedMillis(poolEntry.lastAccessed, now) > aliveBypassWindowMs &&
            isConnectionDead(poolEntry.connection))) {
                closeConnection(poolEntry, poolEntry.isMarkedEvicted() ? EVICTED_CONNECTION_MESSAGE : DEAD_CONNECTION_MESSAGE);
                timeout = hardTimeout - elapsedMillis(startTime);
            }
            else {
                metricsTracker.recordBorrowStats(poolEntry, startTime);
                if (isRequestBoundariesEnabled) {
                    try {
                        poolEntry.connection.beginRequest();
                    } catch (SQLException e) {
                        logger.warn("beginRequest Failed for: {}, ({})", poolEntry.connection, e.getMessage());
                    }
                }
            }
        } while (timeout > 0L);

        metricsTracker.recordBorrowTimeoutStats(startTime);
        throw createTimeoutException(startTime);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new SQLException(poolName + " - Interrupted during connection acquisition", e)
    } finally {
        suspendResumeLock.release();
    }
}
```

최종 반환되는 Connection 객체는
HikariProxyConnection

커넥션 습득 실패 시 예외



Connection의 반납 시

- Connection을 반납하면 다음의 흐름으로 커넥션을 반납 합니다.

1. HikariProxyConnection.close() → 커넥션 요청 시
최종 반환된 커넥션 객체
2. PoolEntry.recycle()
3. HikariPool.recycle()
4. ConcurrentBag.requite(PoolEntry)

Connection의 반납 시 - HikariProxyConnection

```
@Override
public final void close() throws SQLException
{
    // Closing statements can cause connection eviction, so this must run before the conditional below
    closeStatements();

    if (delegate != ClosedConnection.CLOSED_CONNECTION) {
        leakTask.cancel();

        try {
            if (isCommitStateDirty && !isAutoCommit) {
                delegate.rollback();
                LOGGER.debug("{} - Executed rollback on connection {} due to dirty commit state.", poolEntry.getPoolName(), delegate);
            }

            if (dirtyBits != 0) {
                poolEntry.resetConnectionState(this, dirtyBits);
            }

            delegate.clearWarnings();
        }
        catch (SQLException e) {
            // when connections are aborted, exceptions are often thrown that should not reach the application
            if (!poolEntry.isMarkedEvicted()) {
                throw checkException(e);
            }
        }
        finally {
            delegate = ClosedConnection.CLOSED_CONNECTION;
            poolEntry.recycle();
        }
    }
}
```

복잡하지만...
아직 정리되지 않은

1. Statement
2. 트랜잭션 상태
3. 커넥션 상태

정리 해주는 단계

커넥션 풀 반납 과정 시작

Connection의 반납 시 - PoolEntry



```
final class PoolEntry implements IConcurrentBagEntry
{
    private final HikariPool hikariPool;

    void recycle()
    {
        if(connection != null) {
            this.lastAccessed = currentTime();
            hikariPool.recycle(this);
        }
    }
}
```

커넥션 풀에게 자신을 파라미터로
넘기면서 반납 요청

Connection의 반납 시 - HikariPool

1. PoolEntry가 더 이상 재사용되면 안되는 경우

단순하게 커넥션 종료

```
@Override
void recycle(final PoolEntry poolEntry)
{
    metricsTracker.recordConnectionUsage(poolEntry);
    if (poolEntry.isMarkedEvicted()) {
        closeConnection(poolEntry, EVICTED_CONNECTION_MESSAGE);
    } else {
        if (isRequestBoundariesEnabled) {
            try {
                poolEntry.connection.endRequest();
            } catch (SQLException e) {
                logger.warn("endRequest Failed for: {},({})", poolEntry.connection, e.getMessage());
            }
        }
        connectionBag.requite(poolEntry);
    }
}
```

2. ConcurrentBag의 requite() 호출

Connection의 반납 시 - ConcurrentBag



```
public void requite(final T bagEntry)
{
    bagEntry.setState(STATE_NOT_IN_USE);
```

1. 커넥션 사용 끝이기 때문에
상태 STATE_NOT_IN_USE로 원복

```
for (int i = 1, waiting = waiters.get(); waiting > 0; i++, waiting = waiters.get()) {
    if (bagEntry.getState() != STATE_NOT_IN_USE || handoffQueue.offer(bagEntry)) {
        return;
    }
```

```
    else if ((i & 0xff) == 0xff || (waiting > 1 && i % waiting == 0)) {
        parkNanos(MICROSECONDS.toNanos(10));
    }
```

```
    else {
        Thread.yield();
    }
}
```

2. 대기중인 스레드 여부 확인

있을 경우, handoffQueue로 직접 전달

3. 사용대기 중인 곳 없으면 스레드 내에 캐싱 (캐싱 개수 최대 16개 제한)

```
final var threadLocalEntries = this.threadLocalList.get();
if (threadLocalEntries.size() < 16) {
    threadLocalEntries.add(useWeakThreadLocals ? new WeakReference<>(bagEntry) : bagEntry);
}
```

```
}
```


Connection의 반납 시 - ConcurrentBag

```
public void requite(final T bagEntry)
{
    bagEntry.setState(STATE_NOT_IN_USE);
```

1. 반납 시에는 CAS 가 아닌 이유???

```
    for (int i = 1, waiting = waiters.get(); waiting > 0; i++, waiting = waiters.get()) {
        if (bagEntry.getState() != STATE_NOT_IN_USE || handoffQueue.offer(bagEntry)) {
            return;
        }
        else if ((i & 0xff) == 0xff || (waiting > 1 && i % waiting == 0)) {
            parkNanos(MICROSECONDS.toNanos(10));
        }
        else {
            Thread.yield();
        }
    }
}
```

2. 사용 중이지 않은 상태 다시 확인하는 이유??

```
final var threadLocalEntries = this.threadLocalList.get();
if (threadLocalEntries.size() < 16) {
    threadLocalEntries.add(useWeakThreadLocals ? new WeakReference<>(bagEntry) : bagEntry);
}
}
```


Connection의 반납 시 - ConcurrentBag

```
public void requite(final T bagEntry)
```

```
{  
    bagEntry.setState(STATE_NOT_IN_USE);
```

조건식 종료 조건이 대기자가 없을 경우

```
for (int i = 1, waiting = waiters.get(); waiting > 0; i++, waiting = waiters.get()) {
```

```
    if (bagEntry.getState() != STATE_NOT_IN_USE || handoffQueue.offer(bagEntry)) {
```

```
        return;
```

```
    }
```

```
    else if ((i & 0xff) == 0xff || (waiting > 1 && i % waiting == 0)) {
```

```
        parkNanos(MICROSECONDS.toNanos(10));
```

```
    }  
    else {
```

```
        Thread.yield();
```

```
    }
```

```
}
```

↳ 따라서 대기자가 있다면 계속해서
handoffQueue에 전달 시도

↳ i는 재시도 과정 중 휴식 주기 조절 변수로써 사용

↳ 전달 실패시 다른 스레드에게 잠시 기회 넘기고 대기

```
final var threadLocalEntries = this.threadLocalList.get();
```

```
if (threadLocalEntries.size() < 16) {
```

```
    threadLocalEntries.add(useWeakThreadLocals ? new WeakReference<>(bagEntry) : bagEntry);
```

```
}
```

```
}
```


정리

threadLocalList 캐싱 → 빠른 재사용

sharedList → 전체 커넥션 관리

handoffQueue → 직접 전달

threadLocalList → sharedList → handoffQueue 순으로 시도

대기자 있음 → 즉시 전달 (handoff)

대기자 없음 → 캐싱 (thread)

경쟁 최소화 + 재사용 극대화 + 대기 최소화

마무리

- 시간 관계 상 답지 못한 내용이 많습니다...
- 들어주셔서 감사합니다!

